

Order Management System

Database Management System — Week 4 Assignment

STUDENT NAME Yokesh	ENROLLMENT NO. 6253210211062
SUBJECT Database Management System (DBMS)	ASSIGNMENT Week 4 — Project Report

1. Objective

The objective of this project is to design and implement a simple relational database for an Order Management System. The system manages customer product orders by maintaining order-level information (date, customer, total amount) and item-level information (product, quantity, price) separately, following normalized database design principles.

2. Task Description

- Design `Orders` and `Order_Details` tables.
- Manage customer product orders.
- Store order date, quantity, and total amount.
- Perform order insertion and modification operations.
- Generate customer order history reports.

3. Database Design Overview

The system is built around two core tables. The `Orders` table stores one row per customer order (order date, customer reference, and total order amount). The `Order_Details` table stores one row per product line within an order (product, quantity ordered, and unit price), and is linked to `Orders` through a foreign key. This 1-to-many relationship (one `Order` → many `Order_Details`) avoids repeating order-level data for every product and keeps the design normalized.

3.1 Table: Orders

Column	Data Type	Constraint	Description
Order_ID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique order identifier
Customer_ID	INT	FOREIGN KEY (Customers)	Customer who placed the order
Order_Date	DATE	NOT NULL, DEFAULT CURRENT_DATE	Date the order was placed
Total_Amount	DECIMAL(10,2)	NOT NULL, DEFAULT 0.00	Total value of the order
	VARCHAR(20)	DEFAULT 'Pending'	

Column	Data Type	Constraint	Description
Order_Status			Pending / Shipped / Delivered / Cancelled

3.2 Table: Order_Details

Column	Data Type	Constraint	Description
Order_Detail_ID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique line-item identifier
Order_ID	INT	FOREIGN KEY (Orders)	Order this line item belongs to
Product_ID	INT	FOREIGN KEY (Products)	Product being ordered
Quantity	INT	NOT NULL, CHECK (Quantity > 0)	Number of units ordered
Unit_Price	DECIMAL(10,2)	NOT NULL	Price per unit at time of order
Subtotal	DECIMAL(10,2)	NOT NULL	Quantity × Unit_Price

4. SQL — Table Creation

```
-- Reference tables (Customers, Products)
CREATE TABLE Customers (
    Customer_ID INT PRIMARY KEY AUTO_INCREMENT,
    Customer_Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100),
    Phone VARCHAR(15)
);

CREATE TABLE Products (
    Product_ID INT PRIMARY KEY AUTO_INCREMENT,
    Product_Name VARCHAR(100) NOT NULL,
    Price DECIMAL(10,2) NOT NULL
);

-- Orders table
CREATE TABLE Orders (
    Order_ID INT PRIMARY KEY AUTO_INCREMENT,
    Customer_ID INT NOT NULL,
    Order_Date DATE NOT NULL DEFAULT (CURRENT_DATE),
    Total_Amount DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    Order_Status VARCHAR(20) DEFAULT 'Pending',
    FOREIGN KEY (Customer_ID) REFERENCES Customers(Customer_ID)
);

-- Order_Details table
CREATE TABLE Order_Details (
    Order_Detail_ID INT PRIMARY KEY AUTO_INCREMENT,
    Order_ID INT NOT NULL,
    Product_ID INT NOT NULL,
    Quantity INT NOT NULL CHECK (Quantity > 0),
    Unit_Price DECIMAL(10,2) NOT NULL,
    Subtotal DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID) ON DELETE CASCADE,
    FOREIGN KEY (Product_ID) REFERENCES Products(Product_ID)
);
```

5. Order Insertion Operations

```
-- Sample customers and products
INSERT INTO Customers (Customer_Name, Email, Phone)
VALUES ('Rahul Sharma', 'rahul@example.com', '9876543210');

INSERT INTO Products (Product_Name, Price)
VALUES ('Wireless Mouse', 450.00), ('Keyboard', 900.00);

-- Step 1: Create a new order
INSERT INTO Orders (Customer_ID, Order_Date, Total_Amount, Order_Status)
VALUES (1, CURRENT_DATE, 0.00, 'Pending');

-- Step 2: Add product line items to the order (Order_ID = 1)
INSERT INTO Order_Details (Order_ID, Product_ID, Quantity, Unit_Price, Subtotal)
VALUES (1, 1, 2, 450.00, 900.00);

INSERT INTO Order_Details (Order_ID, Product_ID, Quantity, Unit_Price, Subtotal)
VALUES (1, 2, 1, 900.00, 900.00);

-- Step 3: Update the order total from its line items
UPDATE Orders
SET Total_Amount = (
    SELECT SUM(Subtotal) FROM Order_Details WHERE Order_ID = 1
)
WHERE Order_ID = 1;
```

6. Order Modification Operations

```
-- Update quantity of an existing order line item
UPDATE Order_Details
SET Quantity = 3, Subtotal = 3 * Unit_Price
WHERE Order_Detail_ID = 1;

-- Recalculate the order's total after modification
UPDATE Orders
SET Total_Amount = (
    SELECT SUM(Subtotal) FROM Order_Details WHERE Order_ID = 1
)
WHERE Order_ID = 1;

-- Update order status
UPDATE Orders
SET Order_Status = 'Shipped'
WHERE Order_ID = 1;

-- Remove a product line item from an order
DELETE FROM Order_Details
WHERE Order_Detail_ID = 2;

-- Cancel an entire order (Order_Details removed via ON DELETE CASCADE)
DELETE FROM Orders
WHERE Order_ID = 1;
```

7. Customer Order History Reports

7.1 Full order history for a specific customer

```

SELECT
    c.Customer_Name,
    o.Order_ID,
    o.Order_Date,
    p.Product_Name,
    od.Quantity,
    od.Unit_Price,
    od.Subtotal,
    o.Order_Status,
    o.Total_Amount
FROM Orders o
JOIN Customers c ON o.Customer_ID = c.Customer_ID
JOIN Order_Details od ON o.Order_ID = od.Order_ID
JOIN Products p ON od.Product_ID = p.Product_ID
WHERE c.Customer_ID = 1
ORDER BY o.Order_Date DESC;

```

7.2 Order summary (one row per order) for a customer

```

SELECT
    o.Order_ID,
    o.Order_Date,
    o.Order_Status,
    o.Total_Amount,
    COUNT(od.Order_Detail_ID) AS Items_Count
FROM Orders o
LEFT JOIN Order_Details od ON o.Order_ID = od.Order_ID
WHERE o.Customer_ID = 1
GROUP BY o.Order_ID, o.Order_Date, o.Order_Status, o.Total_Amount
ORDER BY o.Order_Date DESC;

```

Sample Report Output (Section 7.2)

Order_ID	Order_Date	Status	Total_Amount	Items_Count
1	2026-08-10	Delivered	1350.00	2
2	2026-08-14	Shipped	900.00	1
3	2026-08-16	Pending	450.00	1

7.3 Total amount spent by each customer (all-time)

```

SELECT
    c.Customer_ID,
    c.Customer_Name,
    COUNT(o.Order_ID) AS Total_Orders,
    SUM(o.Total_Amount) AS Total_Spent
FROM Customers c
JOIN Orders o ON c.Customer_ID = o.Customer_ID
GROUP BY c.Customer_ID, c.Customer_Name
ORDER BY Total_Spent DESC;

```

8. Conclusion

The Order Management System demonstrates key DBMS concepts including normalized table design, primary/foreign key relationships, one-to-many associations, DML operations (INSERT, UPDATE, DELETE), aggregate functions, and multi-table JOIN queries to generate meaningful customer order history reports.